

One brain, every model. Superintelligence that belongs to you.



Overview • PRD • Features • Danav Brain (Rust) • Danav App (Tauri) • Roadmap

---

Overview

Danav is a two-part, local-first cognitive infrastructure and autonomous agent operating system built **100% in Rust (Core Engine)** and **Tauri + TypeScript/React (Native App & Web Control Center)**.

1. **Danav Brain (`danavd` / `danav` CLI):** The headless, single-binary cognitive backend. It acts as the unified hippocampus (relational memory graph + AST code index) and prefrontal cortex (sub-50ms TypeSafe System one decision router + streaming API proxy) for every AI CLI, IDE, and model a user runs.
2. **Danav Application (`Danav App`):** The native cross-platform application built directly on top of Danav Brain. It allows users to pool their existing AI subscriptions, CLI sessions, API keys, and local models into a unified compute layer, create custom autonomous agents, and orchestrate tasks across local codebases, native apps, and cloud services.

Part 1: Product Requirements Document (PRD)

1.1 Problem Statement

Modern developers and power users operate across fragmented AI silos—Claude Code in one terminal, Cursor in an IDE, Gemini CLI in another tab, Codex for background tasks, and local models via Ollama. Every tool suffers from four systemic failures:

1. **Context Amnesia & Siloing:** Architectural decisions, bug fixes, and user preferences learned in one tool are invisible to the others.
2. **Token & Latency Bloat:** Agents repeatedly read 1,500-line files from scratch instead of surgical AST symbol slices, wasting context windows and money.
3. **Generative Routing Waste:** Slow, expensive frontier models are used for trivial classification, routing, and safety checks that should take less than 20ms.
4. **Subscription Lock-In:** Users pay for multiple AI subscriptions and API quotas but lack a single unified workspace to pool those subscriptions and run custom cross-app agents on their own hardware.

1.2 Product Vision & Core Value Proposition

- **One Brain, Every Model:** Every CLI, IDE, and agent reads from and writes to a single local relational memory graph and AST index.
- **Zero-Search Surgical Context:** Turn-1 prompts automatically receive a sub-15ms, 400–600 token `context_pack` containing exact AST symbol line ranges, relevant relational memories, and active test states—cutting token consumption by up to 80%.

- **Sub-50ms TypeSafe System one Routing:** Every internal decision (routing, tool selection, safety gating, memory promotion) uses constrained zero-text primitives (`Choice` , `Score` , `Noul`) with a 0% structural hallucination guarantee.
- **Bring Your Own Subscription (BYOS) Agent OS:** The Danav App turns passive AI subscriptions and local models into proactive, cross-app workers governed by strict local privacy and resource caps.

1.3 Target Personas

- **Persona A — The Polyglot AI Engineer (Danav Brain CLI):** Uses multiple coding agents daily; wants instant context sharing across tools, lower API bills, zero-latency AST slicing, and 100% local data ownership.
- **Persona B — The Autonomous Power User / Builder (Danav App):** Wants to build custom background agents that connect GitHub, Notion, Gmail, Slack, Browser, and local files using their existing AI plans without paying a SaaS middleman.
- **Persona C — Privacy-Conscious Engineering Teams:** Needs peer-to-peer memory and skill syncing across a local network with strict secret redaction and zero cloud lock-in.

1.4 Product Scope & Phased Rollout

Phase	Deliverable	Core Focus	Target Metrics
Phase 1: Core Wedge	`danav` Single-Binary CLI & Proxy	Multi-CLI auto-wiring, OpenAI/Anthropic proxy, Tree-sitter AST slicer, Relational Memory Graph.	`< 15ms` context pack generation; `> 70%` file-read token reduction.
Phase 2: Cognitive Moat	TypeSafe System one + Web UI (`:7420`)	Constrained decision engine (`Choice` / `Score` / `Noul`), local ONNX micro-models, Trace Replay UI, MemoryBench.	`< 50ms` System one routing; `0%` schema/tool hallucination.
Phase 3: Danav App	Native Tauri Application (Desktop & Mobile)	BYO-Subscription pool manager, visual Agent Builder, DAG workflow compiler, OS/Browser automation bridge.	`< 150MB` idle RAM; 1-click agent packaging (`.danav-agent`).

Part 2: The Complete 110-Feature Specification

I. Core Architecture, Multi-CLI & Proxy Layer (Danav Core)

1. **Local-First Single-Binary Daemon:** Runs locally on the user's machine (or private server over a tunnel) as a single compiled Rust executable with zero monthly fee, keeping all files, indexes, and memories on user-owned storage.
2. **Model-Agnostic Context Layer:** Treats Claude, Codex, Gemini, Cursor, and local endpoints as swappable workers sharing one unified brain.
3. **Universal Multi-CLI Auto-Wiring (`danav init`):** Automatically detects installed CLIs and generates synchronized instruction stubs (`CLAUDE.md` , `AGENTS.md` , `GEMINI.md` , `.cursorrules`) plus MCP server configs.
4. **Session Hooks & CLI Wrapper (`danav run <tool>`):** Captures session start, tool calls, file edits, and final outcomes across any CLI via PTY wrapping, even when native hooks aren't supported.

5. **Built-in CLI Memory Disabler:** Automatically turns off isolated tool-specific memories during setup so context never drifts from the central brain.
6. **One-Time History Importer:** Imports existing `.claude` folders, Cursor rules, and local Markdown notes into the unified store on first setup.
7. **Canonical Markdown Exporter:** Stores rules and skills in one canonical Markdown-with-frontmatter format and compiles them out to each CLI's specific config format.
8. **OpenAI/Anthropic-Compatible Proxy:** Intercepts API requests to automatically inject the surgical context brief and record responses with zero added wait for the first streaming token.
9. **Static Prefix-Cache Alignment:** Structures injected system prompts, tool schemas, and project rules in a deterministic byte order so upstream APIs hit their prompt cache on every turn.
10. **Multi-Lane Cost & Capability Router:** Routes simple queries to fast/cheap configured endpoints and automatically escalates to frontier reasoning endpoints when a test, schema check, or confidence gate fails.

II. Relational Memory Graph, Profiles & Recall Engine

11. **Dual Store Architecture (Documents vs. Atomic Memories):** Separates raw ingested Documents (files, logs, PDFs, code, URLs kept for chunk retrieval) from extracted Atomic Memories (self-contained facts that evolve over time).
12. **Ontology-Aware Relational Memory Graph (3 Edge Types):** Connects every new fact to existing memories using three explicit graph relationships:
    - `UPDATES` **(State Mutation & Contradiction Resolution):** When a fact changes, the new memory supersedes the old one while preserving audit history.
    - `EXTENDS` **(Context Enrichment):** Adds detail to an existing memory without invalidating it, keeping both active in retrieval.
    - `DERIVES` **(Pattern Inference):** Infers unstated second-order facts from patterns across multiple sessions and memories.
13. **Inferred Memory Review Queue:** Flags low-confidence `DERIVES` inferences in a review queue so the user can approve or decline them before they become permanent rules.
14. **Automatic User & Project Profiles (Static + Dynamic + Buckets):** Maintains an instant single-call profile per scope containing stable facts, active tasks, and custom topic-scoped profile slices.
15. **Dreaming Modes (Instant vs. Background Consolidation):** Supports immediate synchronous graph extraction when a fact is needed on the next turn, and background batch "dreaming" to consolidate and deduplicate asynchronously.
16. **Temporal Truth & Automatic TTL Forgetting:** Automatically expires transient facts via `forgetAfter` timestamps, filters out casual noise, and suppresses contradicted facts.
17. **Semantic "Forget by Meaning" with Mandatory `dryRun` Preview:** Lets users run `forget_matching(query, dryRun=true)` to semantically preview all memories about a topic before soft-deleting them.
18. **Virtual Memory Filesystem (`DanavFS` Mount):** Exposes a virtual directory where agents can `ls`, `cat`, and semantically `grep` memories alongside a live, auto-generated `profile.md` digest at the root.
19. **Unified 3-Mode Recall API (`memories | chunks | hybrid`):** Searches extracted atomic memories, raw document/code chunks, or both in a single call.
20. **Graph-Attached Recall (`include: { related, documents, forgotten }`):** Optionally attaches parent/child/sibling graph edges, source documents, and historical/forgotten context alongside every search hit.

21. **Multi-Query Rewrite & Cross-Encoder Reranking:** Optionally expands complex queries into multiple search rewrites, merges BM25 + vector hits, and reranks results with a similarity threshold gate.
22. **Rich Metadata Filter Engine:** Filters memories and profile synthesis by exact operators (`eq` , `neq` , `contains` , `notContains` , `gt` , `gte` , `lt` , `lte`) before ranking.
23. **Namespace & Container-Tag Isolation + Merging:** Scopes every memory, profile, and connector to strict namespaces (`user` , `project` , `repo` , `client`) with support for merging or resetting container tags.
24. **Multimodal Ingestion Pipeline:** Extracts, chunks, and indexes Markdown, code, PDFs (with page/diagram descriptions), images, and audio transcripts into the local index.
25. **Auto-Syncing Knowledge Connectors:** Background sync connectors (with webhook/schedule support and `deleteDocuments: true/false` lifecycle control) for GitHub repos, Notion, Google Drive, Gmail, S3, local folders, and a targeted Web Crawler.

III. TypeSafe System one Decision Engine

26. **System one vs. System 2 Split Architecture:** Never wastes a slow generative LLM call on routing, classification, relevance filtering, or safety checks; runs a sub-50ms TypeSafe Decision Layer in front of every generative model.
27. **Zero-Text Constrained Decision Primitives (`Choice` , `Score` , `Noul`):** Every internal brain judgment is framed as one of three strict, schema-locked question types that output numbers/enums only—never free-form text:
    - **`Choice(options[1..255])` :** Picks from an enumerated list and returns a probability distribution plus confidence score.
    - **`Score(rubric[2..10])` :** Evaluates state on an ordered 2–10 level rubric with continuous interpolation.
    - **`Noul` (Binary 0.0–1.0 Probability):** Answers a strict Yes/No predicate as a single probability value.
28. **Single-Pass Parallel Question Evaluation:** Evaluates multiple `Choice` , `Score` , and `Noul` questions against a piece of state in a single parallel forward pass (routing + tool selection + risk check + memory promotion).
29. **0% Structural Hallucination Guarantee:** Constrained decoding and logit-level token masking make off-schema strings, invalid tool names, or fabricated file paths structurally impossible.
30. **Confidence-Threshold Gating & Escalation:** Every `Choice` and `Noul` decision checks a tuned probability bar (e.g., $p \geq 0.80$); anything below automatically escalates to a System 2 reasoning model.
31. **Pre-Decision State Compaction:** Automatically strips unrelated conversation history and noise before passing state to the TypeSafe decision layer so irrelevant context never degrades classification accuracy.
32. **Deterministic Pre-Compute for Math, Counts & Dates:** Computes line counts, timestamps, token budgets, and file sizes deterministically in native Rust code first, then feeds exact values into the TypeSafe decision layer.
33. **Literal Criteria & Structural Invariant Enforcement:** Defines explicit boundary conditions on every boolean/enum gate and enforces logical invariants in code.
34. **Pinned Decision Rubrics & Version Locking:** Locks tuned threshold bars to specific decision-model/classifier versions in a lockfile (`danav-decisions.lock`) so updates never silently shift routing behavior.
35. **Loose-Text-to-Schema Extractor:** Converts messy session transcripts and terminal logs directly into strictly typed JSON/Rust memory schemas, leaving low-confidence fields `null` rather than guessing.

IV. Fast Context Graph & Surgical Code Routing (Zero-Search Engine)

36. **Background Tree-Sitter AST Indexer:** Parses project files on save and git hooks to index every class, function, method, route, interface, and exact `start_line` – `end_line` range in sub-millisecond time.
37. **Compressed File Skeletons (`read_skeleton`):** Stores and serves header-only views (imports, type definitions, function signatures, docstrings—no bodies), cutting file-reading token costs by ~85%.
38. **Bidirectional Memory ↔ Code Symbol Anchors:** Links every memory, architectural rule, and bug fix directly to the exact file paths, AST symbol names, and line ranges it describes.
39. **Call Graph & Import Dependency Map:** Tracks which symbols import, call, or inherit from each other across the repository.
40. **Git Co-Edit & Co-Access Heatmap:** Learns from Git history and past agent sessions which files are almost always modified together and groups them into Co-Change Clusters.
41. **Live Dirty-State & Stack-Trace Watcher:** Watches `git diff` , uncommitted files, and terminal test failures in real time to pre-resolve failing files and line numbers before the user even asks "fix this".
42. **Sub-15ms Surgical `context_pack` Generator:** Combines hybrid memory recall, AST symbol hits, and 1-hop graph links to inject a 400–600 token map of exact files, line ranges, agent notes, and test commands on Turn 1.
43. **AST-Sliced Symbol Reader (`read_symbol`):** Lets the AI read only the 30–50 lines of a target function plus its immediate imports instead of loading a 1,500-line file.
44. **Syntax-Validated Symbol Editor (`edit_symbol`):** Applies edits directly to a target AST symbol and validates syntax via Tree-sitter before writing to disk.
45. **Pre-Edit Blast-Radius Checker (`impact_check`):** Returns all callers, dependent files, co-edit partners, and test suites that could break before the agent modifies a function.

V. Typed Tools, Skills & Sandboxed Plugins

46. **Minimal Default Toolset (5–8 Core Tools):** Exposes only essential surgical tools by default and uses the TypeSafe `Choice` router to preselect task-specific tools so the model isn't overwhelmed.
47. **Negative-Boundary Tool Descriptions:** Writes every tool and skill description in strict `"Use when X, NOT for Y"` format, benchmarked against the evaluation task set.
48. **Lazy-Loaded Skill Packages (`SKILL.md` + `scripts/` + `tests.yaml`):** Keeps only skill names and one-line descriptions in context until `load_skill` pulls the full procedure on demand.
49. **Per-Model Skill Variants:** Allows skills to include model-specific instruction notes (e.g., formatting tweaks for Claude vs. Gemini vs. Codex).
50. **Automated Skill Distillation from Traces:** Detects when a multi-step agent workflow succeeds repeatedly and drafts a reusable, tested skill for user approval.
51. **Skill Regression Testing & Scoring:** Runs each skill's `tests.yaml` against configured models, tracks real-world success rates, flags flaky skills, and blocks regressions.
52. **Manifest-Locked Plugins (`plugin.yaml`):** Bundles tools, skills, and adapters with SHA-256 hash verification, version lockfiles, staged updates, and one-step rollback.
53. **Rootless Process Sandbox (Linux Namespaces / macOS Seatbelt / Containers):** Executes all plugins, `run_python` , and `run_shell` calls under an unprivileged user with no default network access and strict CPU/RAM/time caps.
54. **Output Truncation with URI Pointers:** Caps tool output size in the prompt and stores full logs under a local `danav://logs/<id>` URI pointer so massive stack traces never blow up the context window.

VI. Offline-First Search, Knowledge & Browser Engine

55. **6-Tier Local-First Lookup Cascade:** Resolves queries in order: (1) Response Cache → (2) Local Offline Doc Index → (3) Free Site APIs (GitHub, StackExchange, PyPI, Crates.io) → (4) Targeted Doc Crawler → (5) Self-Hosted SearXNG → (6) Headless Browser.
56. **Offline Documentation & Changelog Indexer:** Maintains a local SQLite FTS5 + vector index of project-relevant library docs, man pages, and changelogs with scheduled background refreshes.
57. **Accessibility-Tree Browser Tools (`browser_*`):** Serves clean, clickable text/accessibility snapshots over MCP by default (fast, low-token, works with text models), escalating to screenshots/vision only for canvas or charts.
58. **Isolated, Allow-Listed Browser Profile:** Runs browser tools in a clean profile with zero personal cookies/logins and requires explicit confirmation before any form submission or purchase.
59. **Citation-Grounded Web Synthesis:** Rewrites web searches into short queries, extracts relevant passages from top results, cites URLs, and treats all fetched web text strictly as untrusted data.

VII. Security, Policy, Privacy & Resource Control

60. **Single-Chokepoint Policy Engine:** Evaluates effective permissions as the strict intersection of $\text{User Scope} \cap \text{Client Scope} \cap \text{Plugin Manifest}$ before any tool, file, or memory operation runs.
61. **Prompt-Injection Firewall (Data vs. Instruction Separation):** Treats all text from web pages, external files, and memories as passive data envelopes that can never autonomously trigger code execution, file deletion, or network sends without confirmation.
62. **Outbound Secret & PII Redactor:** Scans and strips API keys, tokens, `.env` variables, emails, and private paths using deterministic regex + entropy analysis before any context brief is sent to a hosted model API.
63. **Encrypted System Secret Vault:** Stores upstream API keys and connector credentials in an OS-backed encrypted keychain/vault, never in plaintext configs or memory files.
64. **Append-Only Security Audit Log:** Records every permission check, blocked tool call, redaction event, and confirmation prompt in an immutable cryptographically chained log.
65. **Live Per-Component Resource Monitor (`danav status` & UI Panel):** Displays real-time RAM (MB), CPU (%), and Disk (MB) usage broken down by Core Daemon, Local Embedder, Active Sandboxes, Browser instances, and SQLite caches.
66. **One-Click & CLI Resource Cleanup (`danav clean`):** Lets users immediately unload the idle embedding/decision models from RAM, kill lingering sandboxes/browsers, purge expired web/trace caches, and `VACUUM` SQLite without touching core memories.
67. **Hard RAM & Disk Budget Enforcement:** Enforces user-defined `max_ram_mb` and `max_disk_mb` limits with automatic LRU cache eviction and idle worker unloading.

VIII. Observability, Benchmarking & User Experience

68. **Step-by-Step JSONL Request Tracing:** Logs every request ID, CLI client, TypeSafe routing decision, loaded skills, retrieved memory IDs, AST slices read, latency, and token cost.
69. **Interactive Trace Replay Viewer:** Lets users open any past session in the local web or app dashboard (`localhost:7420`) to inspect the exact context pack injected and why the brain routed or answered as it did.
70. **Built-in Memory & Context Evaluation Suite (`/benchmark-context`):** Ships with a 50-task benchmark and memory precision/recall evaluator (covering long-horizon recall, multi-session state mutation, and surgical SWE context) to gate every prompt, router, or skill update.

71. **Local Web & App Control Center:** Provides a visual Memory Graph browser, Pending Inference review queue, Skill manager, Token Savings analytics, and the Live RAM/Disk Cleanup panel.
72. **Zero-Lock-In Markdown Export & 1-Step Wipe:** Exports the entire brain as clean, portable Markdown files and supports encrypted, restore-tested backups (`danav backup`) or complete one-step deletion.

IX. Advanced Agentic Collaboration & Autonomous Task Orchestration

73. **Multi-Agent Shared Workspace Lockfile (`.danav/locks.lock`):** Prevents concurrent local agents (e.g., Claude Code in Terminal 1 and Codex in Terminal 2) from editing the exact same file or AST symbol simultaneously via AST-node-level advisory locks.
74. **Autonomous Background "Sweeper" Agent:** A lightweight local worker that runs during idle times to refactor deprecated code patterns found in recent memories, update project documentation, and clean up stale tests.
75. **Agentic Self-Correction Loop on Test Failure:** When `run_sandbox` catches a test failure, the brain automatically intercepts the stack trace, pulls the relevant AST symbols, and injects a corrective micro-brief without requiring user intervention.
76. **Deterministic Task Checkpoints & Rollback:** Automatically creates a lightweight local git tree snapshot before executing any agent tool call batch, allowing one-click rollback if the model breaks the build.
77. **Cross-Project Memory Transfer ("Cherry-Pick"):** Allows users or agents to safely export and import specific memory subtrees (e.g., sharing a custom authentication pattern from Project A to Project B).
78. **Natural Language Memory & Rule Linter:** A background lint engine that scans Markdown memories and project rules for internal contradictions, syntactic ambiguity, or outdated guidelines and prompts the user to resolve them.
79. **Adaptive Tool Timeout & Retry Backoff:** Dynamically adjusts tool execution timeouts and token budget caps based on historical success rates and latency profiles per tool.
80. **Federated Multi-User Team Sync (Optional Peer-to-Peer):** Allows developers on the same local network or team to securely sync verified project memories and distilled skills peer-to-peer over encrypted mDNS/QUIC channels without relying on a central cloud.
81. **Agent Handoff Protocol ("Session Stashing"):** Lets an agent serialize its current working state, active hypothesis, and memory pointers into a structured JSON artifact so another model (or human) can resume the task instantly.
82. **Incremental AST Diffing for Fast Re-Indexing:** Instead of reparsing whole files on save, uses incremental Tree-sitter parsing to update only modified AST nodes in the database in sub-millisecond time.
83. **Token Cost & Budget Guardian:** Real-time token budget tracking across all connected CLIs with hard spending alerts and automatic fallback to cheaper local/cloud lanes when monthly budgets approach their limit.
84. **Episodic Memory Clustering & Timeline Visualizer:** Automatically groups past debugging sessions and feature additions into chronological "epics" or timelines in the dashboard for quick historical audits.
85. **Custom Schema Injector for Domain-Specific Agents:** Lets users define custom frontmatter schemas for specialized domains (e.g., legal contracts, medical charts, ERP specs, game design docs) without breaking the core brain engine.
86. **Headless IDE & Editor Extensions (VS Code, Neovim, Zed):** Native extensions that stream surgical context briefs directly into the editor status bar, gutter indicators, and inline completion prompts.
87. **Semantic Git Commit Message Generator:** Automatically analyzes staged diffs, linked memories, and System one decision outputs to draft descriptive, standards-compliant commit messages.

38. **Automatic Dependency Vulnerability Watcher:** Indexes local `package.json`, `Cargo.toml`, or `requirements.txt` files and maps them to local advisory databases for instant CVE-aware context packing.
39. **Prompt-Caching Hit Rate Optimizer:** Continuously re-orders static prefixes, tool manifests, and project instructions to maximize cache hit ratios across OpenAI, Anthropic, and Gemini API providers.
40. **Interactive "Why Did You Do That?" Trace Inspector:** A conversational debug tool in the UI where users can query an execution trace (e.g., *"Why did the agent edit file X?"*) and receive a deterministic breakdown of the retrieved memories and AST graph hops.
41. **Ephemeral Sandbox Workspaces:** Spins up isolated, copy-on-write scratchpad directories (`git worktree` / OverlayFS) for agents to test risky refactors or prototype code without touching the main working tree.
42. **Automated Dead Code & Ghost Memory Detection:** Identifies memories pointing to deleted files or symbols and automatically prunes or flags them during the nightly consolidation cycle.
43. **Multi-Modal OCR & Diagram Analyzer:** Automatically parses architecture diagrams, UI mockups, and whiteboard screenshots dropped into the workspace, converting them into structured reference memories.
44. **Configurable Privacy Boundaries (Per-Folder Exclusion):** Allows users to designate specific folders (e.g., `secrets/`, `personal/`) via `.danavignore` as strictly off-limits to both proxy redaction scans and memory extraction.
45. **One-Command Diagnostic Doctor (`danav doctor`):** Automatically diagnoses database corruption, missing Tree-sitter grammars, broken MCP socket connections, and stale lockfiles, offering automatic repairs.

X. Cognitive Plasticity, Speculative Execution & Failure Immunity

46. **Negative-Constraint & Anti-Pattern Ledger ("Scar Tissue Memory"):** Explicitly records failed approaches, rejected libraries, and reverted git commits with higher retrieval weight than normal memories, ensuring no connected model ever repeats the same mistake twice.
47. **Speculative Context Pre-Fetching (Zero-Latency Turn 2):** While the user is reading Turn 1 or typing in the terminal/UI, the System one engine predicts the 3 most likely next AST symbols and memories based on the active call graph and pre-loads them into hot RAM.
48. **Cross-Model Consensus & Adversarial Verification Gate:** For high-risk edits (flagged by a System one `Score` rubric), automatically pipes the proposed diff created by Model A (e.g., Claude) through Model B (e.g., Gemini or Codex) in a blind adversarial review before presenting it to the user.
49. **Counterfactual Replay Simulator (`danav replay --what-if`):** Runs historical session traces from JSONL logs against a newly tweaked System one rubric, new skill, or cheaper model to prove quantitatively whether the new setup solves bugs faster or cheaper.
50. **Decay-Weighted Synaptic Pruning (Hebbian Memory Scoring):** Tracks how often a retrieved memory actually contributed to an accepted git diff or successful tool call; memories that are retrieved often but ignored by the agent lose synaptic weight and sink in ranking.
51. **Semantic Diff Compression for Long Sessions:** Instead of keeping raw turn-by-turn file edit history in the conversation window, collapses intermediate edits into a single mathematical state transition ($State_0 \rightarrow State_N$) to keep long debugging sessions sharp.
52. **Local ONNX/GGUF Micro-Model Hot-Swapper:** Dynamically loads quantized 0.5B–3B models into RAM strictly for the System one decision gates and embedding passes, keeping inference under 20ms on consumer laptops without requiring a dedicated GPU.

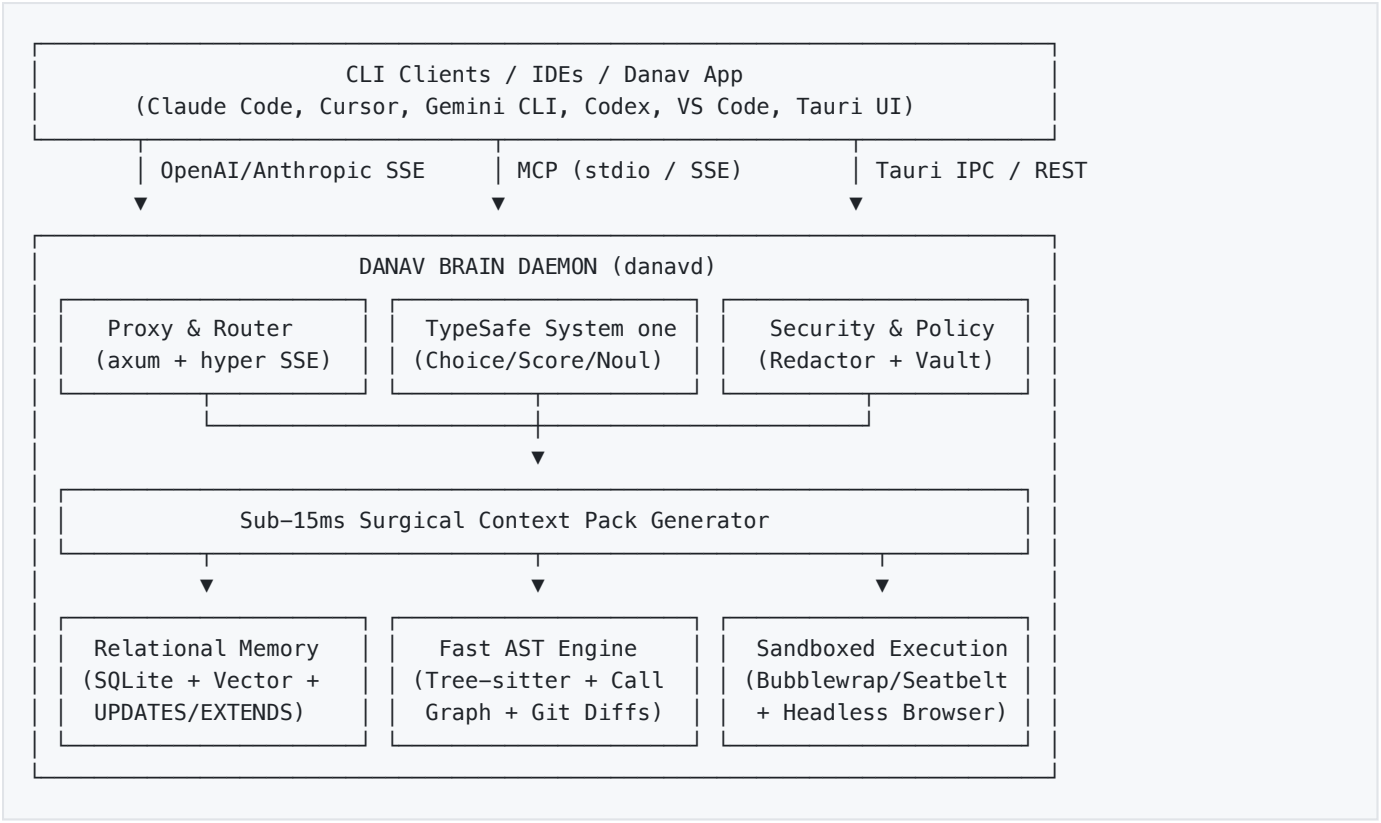
XI. Danav App: Cross-App Orchestration & BYO-Subscription Runtime

03. **Unified Subscription & Quota Pool Manager:** Aggregates the user's existing CLI sessions (Claude Code, Gemini CLI, Codex), API keys, and local models (Ollama/vLLM) into a single "Compute Pool," automatically load-balancing tasks based on remaining rate limits and token quotas.
04. **Event-Driven Reactive Triggers ("When X, Wake Agent Y"):** Turns Danav from a passive request-response brain into an event-driven daemon that wakes up autonomous agents on webhooks, Git pushes, incoming emails, Slack messages, cron schedules, or file system changes.
05. **Universal App-State Graph (OS Accessibility + DOM Bridge):** Extends the Tree-sitter AST concept beyond code to the user's OS—mapping open application windows, active browser tabs, and native OS accessibility trees into a compact text graph so Danav agents can drive non-API applications with low token overhead.
06. **Scoped OAuth & Session Delegator for Third-Party Apps:** Allows users in the Danav UI to grant an agent temporary, capability-scoped access to external apps (e.g., *"Agent can read Jira tickets and draft Notion pages, but cannot delete or send external emails"*).
07. **DAG Workflow Compiler (Natural Language to Deterministic Pipelines):** Lets users describe a multi-app workflow in plain English inside the Danav App once, and compiles it into a deterministic, repeatable Directed Acyclic Graph (DAG) gated by System one `Choice` and `Noul` checkpoints so daily runs cost almost zero LLM tokens.
08. **Human-in-the-Loop Escalation Inbox (Async Approval Queue):** When a background agent hits a low-confidence System one gate ($p < 0.80$) or a high-risk policy boundary while working across apps, it pauses state and pushes a 1-click "Approve / Modify / Reject" card to the Danav App UI and mobile notification channel.
09. **Cross-App Data Lineage & Taint Tracking:** Tags data pulled from untrusted external apps (e.g., an incoming email or public web page) as `tainted`, preventing any downstream agent from passing that payload into a shell command or private database query.
10. **Portable Agent Blueprints (`.danav-agent` Manifests):** Packages an agent's persona, System one decision rubrics, required app connectors, and distilled skills into a single shareable file so users can install or share community-built agents in one click.

Part 3: Technical Requirements Document (TRD) — Danav Brain (Backend)

3.1 Language & Runtime Architecture (100% Rust Core)

To guarantee Feature #1 (Single-Binary Daemon), Feature #26 (Sub-50ms System one), and Feature #42 (Sub-15ms Context Pack), **Danav Brain (`danavd`) is written 100% in Rust**.



3.2 Cargo Workspace Structure

```
danav/
├── Cargo.toml                    # Workspace root
├── crates/
│   ├── danav-cli/                # Binary entry (`danav init`, `danav run`, `danav clean`, `danav do
│   ├── danav-daemon/             # Background daemon (`danavd`), IPC socket, lifecycle & RAM watchdo
│   ├── danav-proxy/              # Axum OpenAI/Anthropic/Gemini proxy, prefix-cache aligner, SSE inj
│   ├── danav-mcp/                # MCP server implementation (`read_symbol`, `edit_symbol`, `recall`
│   ├── danav-system1/            # Constrained logit-masking engine (`Choice`, `Score`, `Noul`) via
│   ├── danav-memory/             # Dual-store engine, relational graph (`UPDATES`/`EXTENDS`/`DERIVES
│   ├── danav-ast/                # Incremental Tree-sitter indexer, skeleton generator, call-graph &
│   ├── danav-pack/               # Sub-15ms `context_pack` assembler, speculative pre-fetcher, token
│   ├── danav-sandbox/            # Rootless execution (Linux userns/bwrap, macOS seatbelt), PTY wrap
│   ├── danav-connectors/         # Native async connectors (GitHub, Notion, Drive, Gmail, S3, Crawle
│   ├── danav-security/           # Intersection policy engine, taint tracker, secret redactor, OS ke
│   └── danav-bench/              # Built-in 50-task evaluation harness & counterfactual trace replay
└── apps/
    └── danav-app/                # Tauri v2 + React/Next.js + Tailwind CSS (serves :7420 & native Da
```

3.3 Core Rust Crate Dependencies

- **Async & HTTP Proxy:** `tokio`, `axum`, `hyper`, `tower-http`, `reqwest`, `eventsource-stream`
- **CLI & Terminal Wrapping:** `clap`, `portable-pty`, `nix`, `crossterm`
- **Storage, Graph & Vectors:** `rusqlite` (bundled with `fts5` + `sqlite-vec`), `r2d2_sqlite`, `blake3`
- **Local Inference & System one Logit Masking:** `ort` (ONNX Runtime for embeddings/reranking/classification) + `llama-cpp-2` (for local GGUF micro-models with custom token logit-bias grammars)

- **AST & Git Intelligence:** `tree-sitter` (plus language grammars), `git2`, `ignore`, `notify` (file system watcher)
- **Security & Vault:** `keyring`, `chacha20poly1305`, `ed25519-dalek`

3.4 Storage & Database Schema (`~/.danav/brain.db` — SQLite WAL Mode)

Using a single embedded SQLite database in `WAL` (Write-Ahead Logging) mode with `mmap_size = 268435456` guarantees sub-millisecond reads, transactional safety, and zero external database processes.

```
-- 1. Raw Documents Store (Files, Docs, Transcripts)
CREATE TABLE documents (
    id TEXT PRIMARY KEY,                  -- BLAKE3 content-addressed ID
    namespace TEXT NOT NULL,              -- e.g., 'project:danav', 'user:global'
    source_uri TEXT NOT NULL,             -- File path or connector URL
    content_hash TEXT NOT NULL,
    mime_type TEXT NOT NULL,
    metadata_json TEXT NOT NULL,          -- Filterable via JSONB operators
    created_at INTEGER NOT NULL,
    updated_at INTEGER NOT NULL
);

-- 2. Atomic Memories Store
CREATE TABLE memories (
    id TEXT PRIMARY KEY,
    namespace TEXT NOT NULL,
    memory_type TEXT NOT NULL,            -- 'fact' | 'rule' | 'scar_tissue' | 'inferred' | 'profile_s
    content TEXT NOT NULL,
    status TEXT NOT NULL DEFAULT 'active',-- 'active' | 'superseded' | 'pending_review' | 'forgotten'
    confidence REAL NOT NULL DEFAULT 1.0,
    hebbian_weight REAL NOT NULL DEFAULT 1.0, -- Synaptic utility score (#100)
    forget_after INTEGER,                 -- Temporal TTL (#16)
    source_doc_id TEXT REFERENCES documents(id),
    created_at INTEGER NOT NULL
);

-- 3. Ontology-Aware Relational Memory Graph Edges
CREATE TABLE memory_edges (
    id TEXT PRIMARY KEY,
    source_id TEXT NOT NULL REFERENCES memories(id),
    target_id TEXT NOT NULL REFERENCES memories(id),
    edge_type TEXT NOT NULL,              -- 'UPDATES' | 'EXTENDS' | 'DERIVES'
    reason TEXT,
    created_at INTEGER NOT NULL
);

-- 4. Tree-Sitter AST Symbol Index & Bidirectional Anchors
CREATE TABLE ast_symbols (
    id TEXT PRIMARY KEY,                  -- repo + file_path + symbol_qname
    repo_id TEXT NOT NULL,
    file_path TEXT NOT NULL,
    symbol_name TEXT NOT NULL,
    symbol_kind TEXT NOT NULL,            -- 'fn' | 'struct' | 'class' | 'interface' | 'route'
    start_line INTEGER NOT NULL,
    end_line INTEGER NOT NULL,
    signature_skeleton TEXT NOT NULL,     -- Header-only representation (#37)
    ast_hash TEXT NOT NULL
);

CREATE TABLE memory_symbol_anchors (
    memory_id TEXT NOT NULL REFERENCES memories(id),
    symbol_id TEXT NOT NULL REFERENCES ast_symbols(id),
    PRIMARY KEY (memory_id, symbol_id)
);
```

3.5 TypeSafe System one Decision Engine Specification

Instead of prompting a generative LLM with *"Please respond in JSON,"* `danav-system1` intercepts state and evaluates constrained primitives in a single forward pass using logit masking over a local quantized model (e.g.,

Qwen-2.5-0.5B/1.5B-Instruct or a fine-tuned ModernBERT classifier via ONNX):

1. **`Choice(options: &[&str]) ->ChoiceResult`:**
   - Masks the vocabulary softmax layer exclusively to the token sequences representing `options[0..N]`.
   - Computes exact normalized sequence probabilities:

$$P(o_i \mid S) = \frac{\exp(\text{logit}(o_i))}{\sum_{j=1}^{N} \exp(\text{logit}(o_j))}$$

```
* If $\max_i P(o_i \mid S) < \tau_{\text{choice}}$ (pinned in `danav-decisions.lock`, default `0.80`),
```

2. **`Score(rubric: &Rubric) -> f32`:**
   - Masks logits to integer tokens `[1..K]` corresponding to explicit literal criteria definitions.
   - Computes the expected continuous score via probability-weighted interpolation:

$$\text{Score}(S) = \sum_{k=1}^{K} k \cdot P(k \mid S)$$

3. **`Noul(predicate: &str) -> f32`:**
   - Evaluates a strict boolean invariant by masking output exclusively to `{Yes, No}` tokens:

$$P(\text{Yes} \mid S) = \frac{\exp(\text{logit}(\text{Yes}))}{\exp(\text{logit}(\text{Yes})) + \exp(\text{logit}(\text{No}))}$$

3.6 Sub-15ms `context_pack` Pipeline (Turn-1 Injection)

When a user sends a prompt in Claude Code, Cursor, or the Danav App, `danav-proxy` or `danav-mcp` executes the following pipeline under a strict **15ms wall-clock budget**:

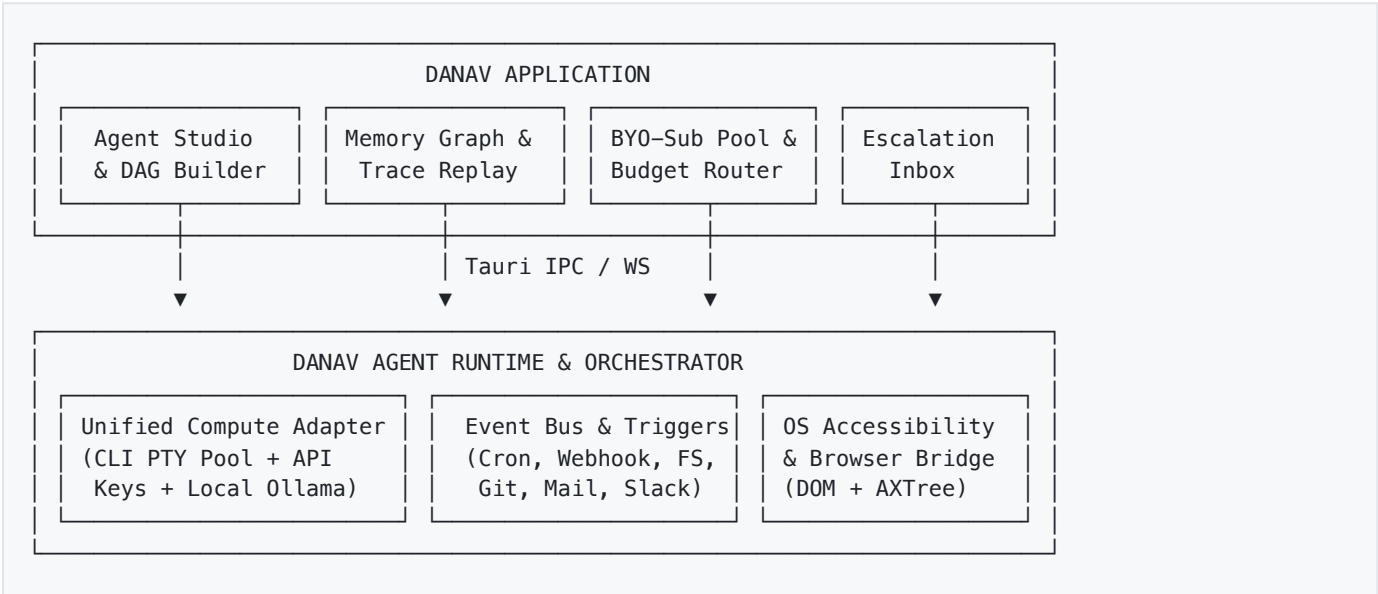
1. **0–2ms (Deterministic Pre-Compute):** Reads dirty `git status`, active terminal stack trace (#41), and open file cursors from the in-memory watcher.
2. **2–7ms (Parallel Hybrid Recall + AST Lookup):**
   - Runs FTS5 BM25 + quantized `sqlite-vec` search across active memories (`status = 'active'`).
   - Traverses 1-hop `UPDATES` / `EXTENDS` edges and `memory_symbol_anchors` to pull linked AST symbols.
   - Queries `ast_symbols` for compressed skeletons (`read_skeleton`) of co-edited files (#40).
3. **7–12ms (System one Filtering & Compaction):** Runs a parallel `Noul` relevance pass and strips any secret/PII strings (#62).
4. **12–15ms (Prefix-Aligned Injection):** Formats the deterministic 400–600 token XML/Markdown `<danav_context_pack>` block and appends it after the static cached system prefix (#9).

Part 4: Technical Requirements Document (TRD) — Danav Application (Frontend & Agent OS)

4.1 Application Architecture (Tauri v2 + React + Rust Core)

Rather than building two separate codebases for the local web dashboard (`localhost:7420`) and the standalone **Danav Application** (**Danav App**), both are built from a single `apps/danav-app` frontend wrapped with **Tauri v2**:

- **Headless Server Mode (`danav ui`):** `danavd` serves the compiled static frontend bundle directly on `[http://127.0.0.1:7420](http://127.0.0.1:7420)` over WebSockets/REST.
- **Native App Mode (`Danav.app` , `.apk` , `.ipa` , `.exe`):** Tauri embeds `danav-daemon` directly into the native binary, communicating via zero-copy Tauri IPC commands and managing system hooks, global shortcuts, and accessibility permissions.



4.2 Unified BYO-Subscription & Compute Pool Engine (#103)

Users should never have to pay Danav a markup on tokens when they already pay for AI tools. Danav App implements three compliant adapter lanes inside the **Unified Compute Pool**:

1. **Official CLI Session Lane (Zero API Cost):** Spawns and pools persistent headless sessions of installed official CLIs (`claude` , `gemini` , `codex`) inside managed PTYs (`portable-pty`), routing agent subtasks through the user's existing CLI plans while respecting rate limits.
2. **Direct API Key Lane (BYOK):** Connects directly to Anthropic, OpenAI, Google Gemini, Groq, and OpenRouter keys stored in the OS Keychain (#63), applying automatic prefix caching (#9) and budget ceilings (#83).
3. **Local Inference Lane (Zero Cloud Cost):** Auto-discovers local Ollama, LM Studio, vLLM, and embedded `llama.cpp` endpoints for offline coding, privacy-restricted folders (#94), and background Sweeper tasks (#74).

4.3 Deterministic DAG Workflow Compiler (#107)

When a user creates an agent in Danav (e.g., *"Every morning at 9 AM, check my GitHub PR reviews, cross-reference failing CI logs with our local codebase, draft a fix in an ephemeral worktree, and ping me on Slack with an approval card"*):

1. **Compilation Step:** A System 2 model translates the natural language prompt into a typed `workflow.danav.json` Directed Acyclic Graph (DAG).
2. **Execution Step:** Every node transition in the DAG is governed by a deterministic Rust tool call or a sub-50ms System one `Choice` / `Noul` gate. Generative LLMs are invoked **only** at nodes that require synthesis or code generation.
3. **Taint Enforcement (#109):** Every edge in the DAG carries a `SecurityLabel { integrity: Trusted | Untrusted, confidentiality: Public | Private }`. If a node ingests an external email or web page (`Untrusted`), the Policy Engine (#60) blocks any downstream node in that execution trace from invoking `run_shell`, `edit_symbol`, or outbound network POSTs unless the user clicks **Approve** in the Escalation Inbox (#108).

4.4 Universal App-State Graph: OS & Browser Automation (#57, #105)

To let Danav agents interact with any desktop or mobile application (Slack, Figma, Linear, Notion, Excel) without burning thousands of vision tokens on screenshots:

- **macOS (`AXUIElement`) / Windows (`UIAutomation`) / Linux (`AT-SPI2`):** A native Rust OS bridge queries the active window's accessibility tree and serializes interactive elements (buttons, text fields, tables, menus) into a compact, numbered Markdown/DOM snapshot (`<button id="@e14">Submit</button>`).
- **Playwright / CDP Bridge:** Attaches to an isolated browser profile (#58) and extracts the accessibility tree. Vision models are called only when a System one `Noul("contains_canvas_or_chart")` check returns $p \geq 0.80$.

4.5 Portable `.danav-agent` Blueprint Format (#110)

Agents created in the Danav App can be exported and shared as a single signed archive or declarative manifest:

```
apiVersion: danav.si/v1
kind: AgentBlueprint
metadata:
  name: pr-triage-and-fix
  version: 1.2.0
  sha256: "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855"
spec:
  compute_lane_preference: ["local_system1", "cli:claude", "api:gemini-2.5-pro"]
  triggers:
    - type: github_webhook
      events: ["pull_request.opened", "check_suite.failed"]
  permissions:
    filesystem: ["read:repo", "write:ephemeral_worktree"]
    connectors: ["github:read", "slack:post_draft"]
    network_egress: false
  system1_gates:
    - id: requires_human_review
      primitive: Score
      rubric_ref: "blast_radius_severity_v1"
      escalate_above: 6.5
  skills:
    - "skills/rust-test-healer"
    - "skills/git-bisect-surgical"
```

Part 5: Non-Functional Requirements & Performance SLAs

Metric	Target SLA	Enforcement Mechanism
Single-Binary Size (`danav`)	`< 35 MB` (stripped release binary)	LTO (`fat`), `codegen-units = 1` , `strip = true` , dynamic model download on first run.
Turn-1 `context_pack` Latency	`< 15 ms` (P95)	Pre-indexed Tree-sitter SQLite tables + in-memory git diff watcher + quantized vectors.
System one Decision Pass (`Choice` / `Score` / `Noul`)	`< 45 ms` (P95 on CPU)	Quantized INT8 ONNX Runtime + single-pass batch evaluation (#28).
Proxy First-Token Overhead	`< 5 ms` over raw upstream API	Zero-copy `hyper` SSE streaming + asynchronous post-response memory extraction.
Idle Daemon RAM Footprint	`< 60 MB` (Brain CLI) / `< 150 MB` (Danav App)	Automatic LRU cache eviction and unloading of idle ONNX models via `danav clean` (#66, #67).
Structural Hallucination Rate	`0.0%` on internal routing & tool selection	Hard token-level logit masking in `danav-system1` (#29).

Part 6: Concrete Engineering Roadmap (16-Week Execution Plan)

- **Weeks 1–4 (Milestone 1 — The Surgical Code & Memory Core):**
  - Scaffold the Rust workspace (`danav-cli` , `danav-daemon` , `danav-ast` , `danav-memory`).
  - Implement the SQLite WAL schema, Tree-sitter background indexer (#36), `read_skeleton` (#37), `read_symbol` (#43), and `edit_symbol` (#44).
  - Build the Dual Store and `UPDATES` / `EXTENDS` / `DERIVES` relational memory graph (#11–12).
  - Ship `danav init` (#3) and the MCP server so Claude Code, Cursor, and Gemini CLI immediately share one brain.
- **Weeks 5–8 (Milestone 2 — Proxy, System one & Security Engine):**
  - Build `danav-proxy` with prefix-cache alignment (#8, #9) and the Outbound Secret Redactor (#62).
  - Integrate `ort` / `llama-cpp-2` in `danav-system1` for constrained `Choice` , `Score` , and `Noul` primitives (#26–35).
  - Wire the Sub-15ms `context_pack` generator (#42) and Rootless Process Sandbox (#53).
- **Weeks 9–12 (Milestone 3 — Local Web Control Center & Benchmarks):**
  - Launch `localhost:7420` UI (#71) featuring the Interactive Trace Replay Viewer (#69), Visual Memory Graph, and Live Resource Cleanup panel (#65–66).
  - Implement the 50-task `/benchmark-context` suite (#70) and Counterfactual Replay Simulator (#99).
- **Weeks 13–16 (Milestone 4 — Danav App & Agent OS):**
  - Wrap the web UI and Rust daemon into the native **Tauri v2 Application (Desktop & Mobile)**.
  - Build the Unified BYO-Subscription Pool Manager (#103), Event-Driven Triggers (#104), DAG Workflow Compiler (#107), OS Accessibility Bridge (#105), and `.danav-agent` packaging (#110).